

Projekt OOPventure

OOP gamifiziert

8. Tag des Informatikunterrichts, 27.03.2026

*Johannes Harz
Cusanus Gymnasium St. Wendel
#gerneperdu*



Ablauf

Was ist OOPventure

Die 3 Phasen

KI-Einsatz

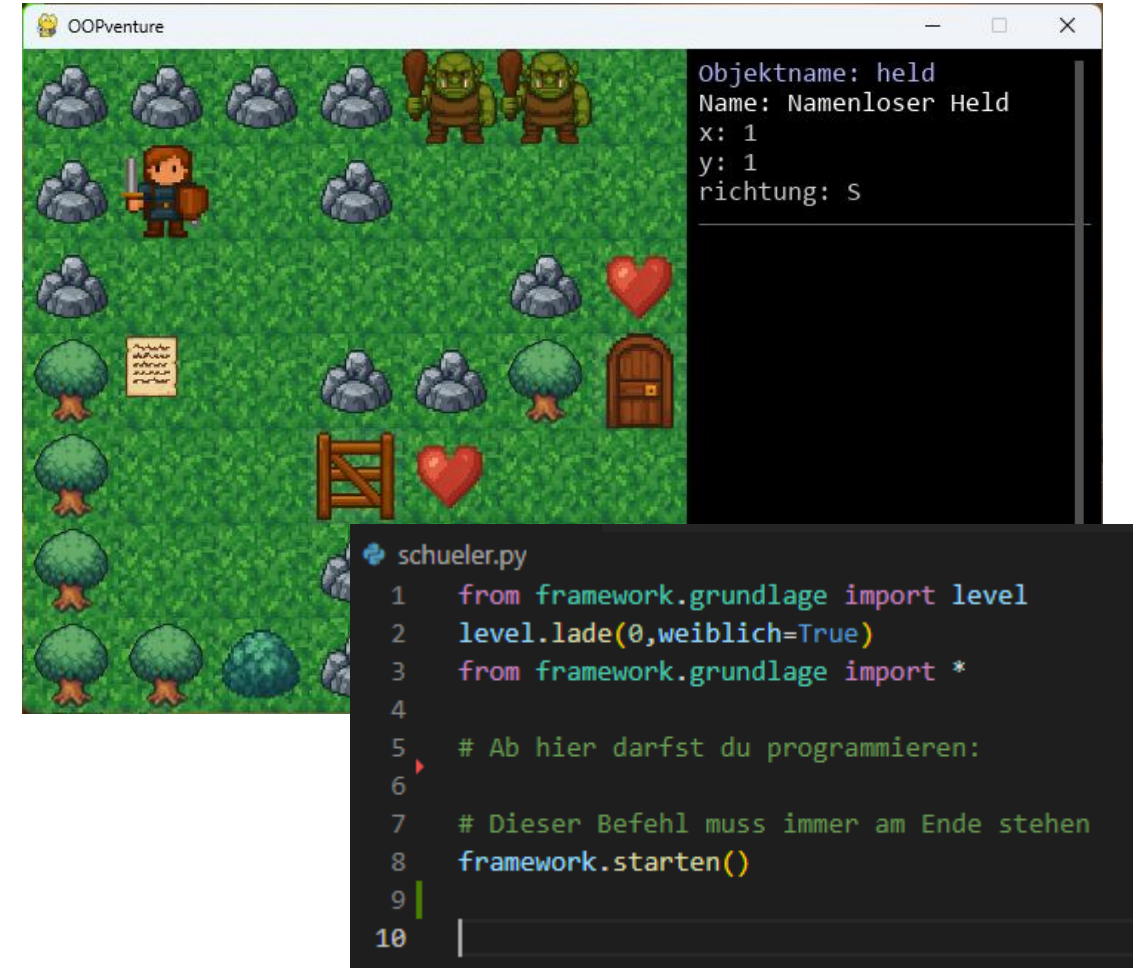
Praxisphase

Ausblick & Abschließende Diskussion

OOPventure

Objektorientierte Programmierung gamifiziert unterrichten

- Lernumgebung für die SEK II
- Abbildung des saarländischen Lehrplans (LK, 2019)
- In jeder Python IDE nutzbar
- Nur 4 Zeilen „Overhead“ in der `schueler.py`
- Python voll nutzbar
- Pygame als Basis vollständig verborgen
- Aufteilung in **3 Phasen**:
 - 1) Einführung & **Algorithmik**
 - 2) **Objekte** nutzen
 - 3) **Klassen** implementieren



Phase 1

Einführung in das Framework & Algorithmik

- Level 1 bis 19 (plus Zusatzlevel)
- **Ziel:** Alle Herzen sammeln
- Ähnlich zu anderen Lernumgebungen
 - JWINF
 - Kara
 - ...
- Fokus auf Algorithmik
- Schrittweise Einführung versch. Hilfsmethoden
 - `was_ist_vorn()`
 - `ist_auf_herz()`
 - ...



```
schueler.py > ...
1  from framework.grundlage import level
2  level.lade(5, weiblich=True)
3  from framework.grundlage import *
4
5  # Ab hier darfst du programmieren:
6  for i in range(3):
7      held.geh()
8      held.nimm_herz()
9
10 held.links()
11 held.geh()
12 held.nimm_herz()
13
14 # Dieser Befehl muss immer am Ende stehen
15 framework.starten()
16
17
```

Phase 2

Konzepte der Objektorientierung nutzen

- Bisheriger Fokus: Algorithmik
- Neuer Fokus: Level durch **Konzepte der OOP** lösen
- Nicht bei gängigen Lernumgebungen verfügbar
- Realität:
 - Objektreferenzen nicht verfügbar
 - Arbeit mit Objekten über Methoden
 - Benötigte Objekte existieren nicht
- Notwendigkeit und Sinn von Datenkapselung
- Unterscheidung zwischen Nutzer- und Entwickleransicht
- Klassische Fehlvorstellung:
 - „Das Leben des Helden muss privat sein, damit es nicht manipuliert werden kann.“
 - Entwickler hat vollen Zugriff ; Nutzer keinen Zugriff auf Code!



The screenshot shows the OOPventure game window. The game area displays a grid of trees with a hero character on the left and a helper character on the right. A red heart icon is visible in the bottom left corner. The right panel shows the object state for 'held' and 'helferlein'. The bottom panel shows the Python code for the game.

```
Objektname: held
Name: Namenloser Held
x: 0
y: 2
richtung: S

Objektname: helferlein
Name: Fridolin fehlschlag
x: 2
y: 3
richtung: S
```

```
schueler.py > ...
1  from framework.grundlage import level
2  level.lade(21,weiblich=True)
3  from framework.grundlage import *
4
5  # Ab hier darfst du programmieren:
6  helferlein = held.gib_knappe()
7  helferlein.geh()
8  blatt = helferlein.gib_objekt_vor_dir()
9  tuer = held.gib_objekt_vor_dir()
10 text = blatt.gib_spruch()
11 tuer.spruch_anwenden(text)
12
13 # Dieser Befehl muss immer am Ende stehen
14 framework.starten()
15
16
```


Phase 3

Eigene Klasse implementieren

- Klassischer Unterrichtsgang beginnt hier
- Umgang mit abstrakten Objekten
- Klasse zuerst, dann das konkrete Objekt
- In OOPventure: Klassen nach Objekten
- Beginn ab Level 35
- Schrittweise Re-Implementierung ausgewählter Klassen
- Einzigartig(?)
- Rendering durch Framework
- Fokus auf die Klassen und Kern Spiellogik
- Klassen greifbar machen

```
1 class Konto:
2     def __init__(self, inhaber, kontostand=0):
3         self.inhaber = inhaber
4         self.kontostand = kontostand
5
6     def ueberweisen(self, ziel_konto, betrag):
7         if betrag > self.kontostand:
8             print("Nicht genügend Guthaben für Überweisung!")
9         else:
10            self.auszahlen(betrag)
11            ziel_konto.einzahlen(betrag)
```

Zielkonto ist kein konkretes Objekt!



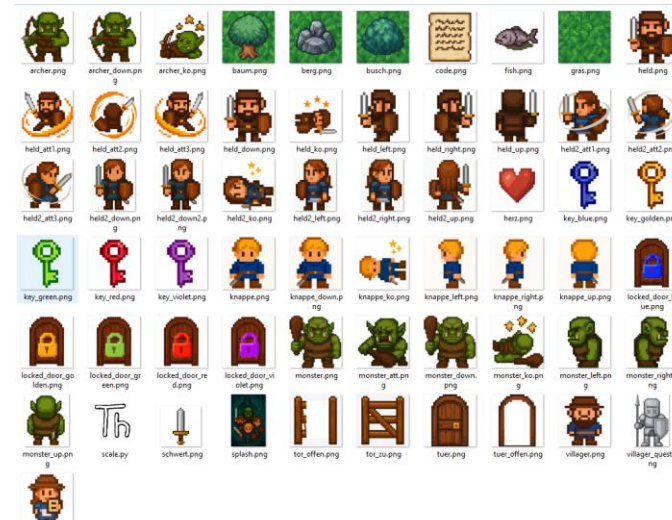
```
1 from framework.grundlage import level
2 level.lade(35, weiblich=True)
3 from framework.grundlage import *
4
5 # Ab hier darfst du programmieren:
6 class Held:
7     def __init__(self, x, y, richtung, weiblich):
8         self.x = x
9         self.y = y
10        self.richtung = richtung
11        self.name = "Bert IV von Schnarchhausen"
12
13 # Dieser Befehl muss immer am Ende stehen
14 framework.starten()
15
16
```

Technisches

Umsetzung & KI-Einsatz

- Umsetzung nur dank **starkem KI-Einsatz** möglich
- Eigener Prototyp:
 - 200 Zeilen Code
 - Unübersichtlich für SuS
- **OOPventure:**
 - 10.000+ Zeilen Codebasis
 - 6.000+ Zeilen Leveldateien
- Alle Grafiken KI generiert (über ChatGPT)
- Codebasis KI generiert (Claude Sonnet & Opus)
- Vibe-Coding mit VS Code & Copilot
- Unterrichtsmaterial ohne KI-Einsatz
- Paralleles Arbeiten

```
36 def links(self):
37     if self.r == "N":
38         self.r = "W"
39     elif self.r == "O":
40         self.r = "N"
41     elif self.r == "S":
42         self.r = "O"
43     else:
44         self.r = "S"
45
46
47 #####
48
49 #####
50 ## AB HIER NICHTS VERÄNDERN! ##
51 ## FRAMWORK! ##
52 #####
53 import pygame,sys, time
```



Loslegen

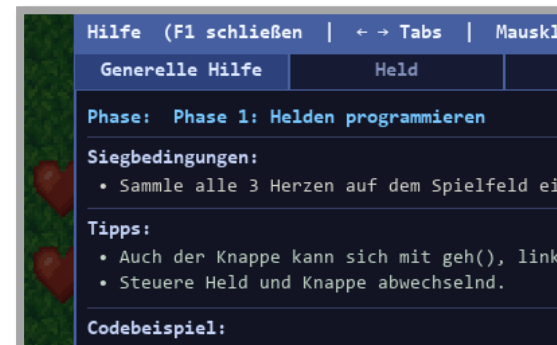
Leitfaden für Installation und Start

- Herunterladen des aktuellen „builds“: <https://t1p.de/eucri>
- Repo-Link: <https://t1p.de/3cf01>
- Entpacken
- In **beliebiger IDE** **schueler.py** bearbeiten
- Optional: pygame installieren
- Wahl des Levels über `level.lade(<levelnummer>)`
- Alten Code löschen oder sichern, z.B. `level1.py`
- **Level 0** führt Spielelemente ein
→ Lösung per Tastatur
- **Ab Level 1:** Lösungen per Code
- **Aktueller Stand:** Lernumgebung benötigt **Begleitmaterial**
- **F1** zeigt Hilfefenster und Befehlsliste



```
1 from framework.grundlage import level
2 level.lade(0, weiblich=True)
```

```
3 from framework.grundlage import *
4
5 # Ab hier darfst du programmieren:
6 held.geh()
7 held.nimm_herz()
8
9 # Dieser Befehl muss immer am Ende stehen
10 framework.starten()
```



Repo:



Praxistest

Ausprobieren des Frameworks & Materialsichtung

Hilfestellungen, Befehle & Siegbedingungen: **F1-Taste**

Ausgewählte Level:

- **Level 0** *(per Tastatur, vgl. AB 1)*
- **Level 1** *(per Code, vgl. AB 1)*
- **Level 9** *(Schleifen & Verzweigung, vgl. AB 4)*

- **Level 21** *(Objekte referenzieren, vgl. AB 6)*
- **Level 34** *(Objekte erzeugen, vgl. AB 12)*

- **Level 35** *(Held implementieren, vgl. AB 13)*
- **Level 37** *(Bewegung implementieren, vgl. AB 13)*

Fertig?

Teste weitere Level oder begutachte das Arbeitsmaterial

Repo:



<https://t1p.de/3cf01>

Held

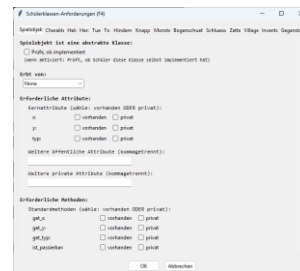
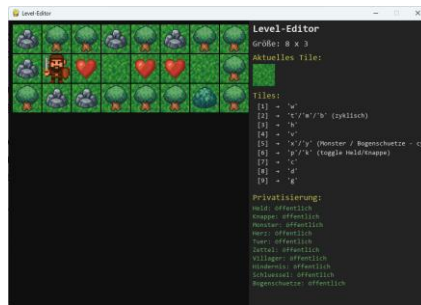
```
x : Integer  
y : Integer  
richtung : String  
name : String  
typ : String  
weiblich : Boolean
```

```
Held(xp,yp : int, richtung : str, w : bool)  
geh()  
links()  
rechts()  
zurueck()
```

Material & Leveleditor

Eine vollständige & erweiterbare Unterrichtsreihe für den Informatik Leistungskurs

- Parallel zum Framework 61-seitiges **Arbeitsmaterial**
- Ausgelegt als Arbeitsblätter zum händischen Ausfüllen
- Zusätzliche Level können mit dem **Leveleditor** erstellt werden oder bestehende Level angepasst werden
- Anpassung der angezeigten Hilfen über den Editor
- Ausführliche Einstellungen der Levelvorgaben
- Aufrufen dieser Einstellungen über F1 bis F5
- S ruft das Speicher-Menü auf
- O öffnet ein bestehendes Level



Abschluss

Ausblick & Diskussion

- **Bugfixing**
- Flexiblere Reaktionen auf Fehler
- Ausbauen & Vervollständigung der Hilfefunktion für alle Level
- Überarbeitung von Arbeitsblatt 2 und 3
- Vollständige Anleitung zur Vervollständigung des Frameworks
- Vollständige Neugestaltung der Grafiken durch Arbeiten der Schülerschaft
- Außergewöhnliche neue Klassen & Objekte



Diskussion: Sinnvolles Tool oder nicht?

Vielen Dank für eure Aufmerksamkeit!

